

1. Nombre de la base de datos

Nombre: Sistema_ADV

Motor: Microsoft SQL Server

Herramienta de administración: SQL Server Management Studio (SSMS)

Sistema_ADV corresponde a una base de datos para la gestión de alquiler de vehículos. Su estructura organiza los usuarios, el inventario de vehículos, las reservaciones, los alquileres efectuados y los pagos asociados. Además de las tablas principales, incorpora vistas, triggers y procedimientos almacenados para automatizar operaciones frecuentes y mantener la integridad de la información.

2. Tablas de la base de datos

La base de datos está formada por cinco tablas principales. Cada tabla representa una entidad del proceso de alquiler y se relaciona con las demás mediante claves primarias y foráneas.

2.1 Tabla usuarios

Centraliza la identidad de las personas que utilizan el sistema. El campo rol permite distinguir entre administradores y clientes. La tabla es referenciada tanto desde reservaciones como desde alquileres.

Campo	Tipo de dato	NULL	Clave/Regla	Predeterminado	Función
id_usuario	INT IDENTITY(1,1)	No	PK	Autonumérico	Identificador único del usuario.
nombre	VARCHAR(100)	No	—	—	Nombre del usuario o cliente.
correo	VARCHAR(100)	No	UNIQUE	—	Correo electrónico; no se permite duplicar.
contrasena	VARCHAR(255)	No	—	—	Contraseña almacenada por la aplicación.
cedula	VARCHAR(20)	Sí	UNIQUE*	NULL	Cédula opcional; si se informa debe ser única.
telefono	VARCHAR(20)	Sí	—	NULL	Teléfono de contacto.
rol	VARCHAR(20)	No	CHECK	Cliente	Solo Administrador o Cliente.

2.2 Tabla vehiculos

Representa el inventario de vehículos. La placa se utiliza como clave primaria y el estado permite controlar si el vehículo está disponible, alquilado o en mantenimiento.

Campo	Tipo de dato	NULL	Clave/Regla	Predeterminado	Función
placa	VARCHAR(10)	No	PK	—	Identificador único del vehículo.
categoria	VARCHAR(50)	Sí	—	NULL	Categoría del vehículo.
marca	VARCHAR(50)	Sí	—	NULL	Marca del vehículo.
modelo	VARCHAR(50)	Sí	—	NULL	Modelo del vehículo.
anio	INT	Sí	—	NULL	Año del vehículo.
precio_dia	DECIMAL(10,2)	No	—	—	Tarifa diaria del alquiler.
estado	VARCHAR(20)	No	CHECK	disponible	Disponible, alquilado o mantenimiento.
foto_url	VARCHAR(255)	Sí	—	NULL	Dirección de la imagen del vehículo.

2.3 Tabla reservaciones

Registra las solicitudes previas al alquiler. Conecta al cliente, al usuario que gestiona la operación y al vehículo seleccionado, además de las fechas previstas y el estado de la reserva.

Campo	Tipo de dato	NULL	Clave/Regla	Predeterminado	Función
no_reserva	INT IDENTITY(1,1)	No	PK	Autonumérico	Número de la reservación.
id_cliente	INT	No	FK	—	Cliente que realiza la reserva.
id_usuario	INT	No	FK	—	Usuario que registra o gestiona la reserva.
placa	VARCHAR(10)	No	FK	—	Vehículo reservado.
fecha_recogida	DATE	No	—	—	Fecha prevista de recogida.
fecha_retorno	DATE	No	—	—	Fecha prevista de retorno.
estado_reserva	VARCHAR(20)	No	CHECK	pendiente	Pendiente, confirmada o cancelada.

2.4 Tabla alquileres

Registra la ejecución real del alquiler. Conserva el cliente, el usuario gestor, el vehículo y las fechas efectivas. El monto_total se calcula automáticamente al insertar un alquiler.

Campo	Tipo de dato	NULL	Clave/Regla	Predeterminado	Función
id_alquiler	INT IDENTITY(1,1)	No	PK	Autonumérico	Identificador del alquiler.
no_reserva	INT	Sí	FK	NULL	Reservación de origen, si existe.
id_cliente	INT	No	FK	—	Cliente responsable del alquiler.
id_usuario	INT	No	FK	—	Usuario que registra el alquiler.
placa	VARCHAR(10)	No	FK	—	Vehículo alquilado.
fecha_inicio	DATE	No	—	—	Inicio del alquiler.
fecha_fin	DATE	No	—	—	Fin del alquiler.
monto_total	DECIMAL(10,2)	Sí	—	NULL	Monto calculado automáticamente por trigger.

2.5 Tabla facturas

Almacena los pagos asociados a los alquileres. Cada registro conserva el monto, el método de pago y la fecha de emisión generada por SQL Server.

Campo	Tipo de dato	NULL	Clave/Regla	Predeterminado	Función
no_factura	INT IDENTITY(1,1)	No	PK	Autonumérico	Número de factura.
id_alquiler	INT	No	FK	—	Alquiler asociado al pago.
monto_pagado	DECIMAL(10,2)	No	—	—	Monto pagado.
metodo_pago	VARCHAR(20)	Sí	CHECK	NULL	Efectivo, tarjeta o transferencia.
fecha_emision	DATETIME2	No	DEFAULT	SYSDATETIME()	Fecha y hora automática de emisión.

3. Descripción y funciones del modelo

El diseño separa las distintas etapas del negocio para evitar mezclar datos de naturaleza diferente. usuarios identifica a las personas; vehiculos mantiene el catálogo disponible; reservaciones representa la intención de alquilar; alquileres registra la operación ejecutada; y facturas documenta el pago. Esta separación permite consultar cada proceso de forma independiente y, al mismo tiempo, reconstruir el ciclo completo de una operación mediante las relaciones entre tablas.

El modelo también incorpora automatización. Al insertar un alquiler, un trigger cambia el estado del vehículo a alquilado y otro calcula el monto_total a partir de la diferencia de días y del precio_dia. Las vistas

simplifican consultas de uso frecuente y los procedimientos almacenados encapsulan operaciones como registrar clientes o consultar vehículos disponibles.

4. Relaciones entre las tablas

Las relaciones se implementan mediante claves foráneas. SQL Server impide que una fila dependiente haga referencia a un registro inexistente, lo que protege la integridad referencial del sistema.

usuarios → **reservaciones (id_cliente)**: Un usuario identificado como cliente puede aparecer en múltiples reservaciones. Relación 1:N.

usuarios → **reservaciones (id_usuario)**: Un usuario que gestiona reservas puede registrar múltiples reservaciones. Relación 1:N.

vehiculos → **reservaciones**: Un vehículo puede ser reservado en diferentes operaciones a lo largo del tiempo. Relación 1:N.

reservaciones → **alquileres**: alquileres.no_reserva puede apuntar a una reservación. Como no_reserva no es UNIQUE dentro de alquileres, el esquema actual permite técnicamente más de un alquiler por una misma reserva. Relación implementada 1:N.

usuarios → **alquileres (id_cliente)**: Un cliente puede tener múltiples alquileres. Relación 1:N.

usuarios → **alquileres (id_usuario)**: Un usuario gestor puede registrar múltiples alquileres. Relación 1:N.

vehiculos → **alquileres**: Un vehículo puede participar en múltiples alquileres en momentos diferentes. Relación 1:N.

alquileres → **facturas**: Cada factura pertenece a un alquiler. Como id_alquiler no tiene UNIQUE en facturas, el diseño admite múltiples facturas para el mismo alquiler. Relación implementada 1:N.

5. Diagramas

5.1 Diagrama Entidad–Relación

El diagrama siguiente muestra las cinco entidades principales, las claves primarias, las claves foráneas y las conexiones entre ellas.

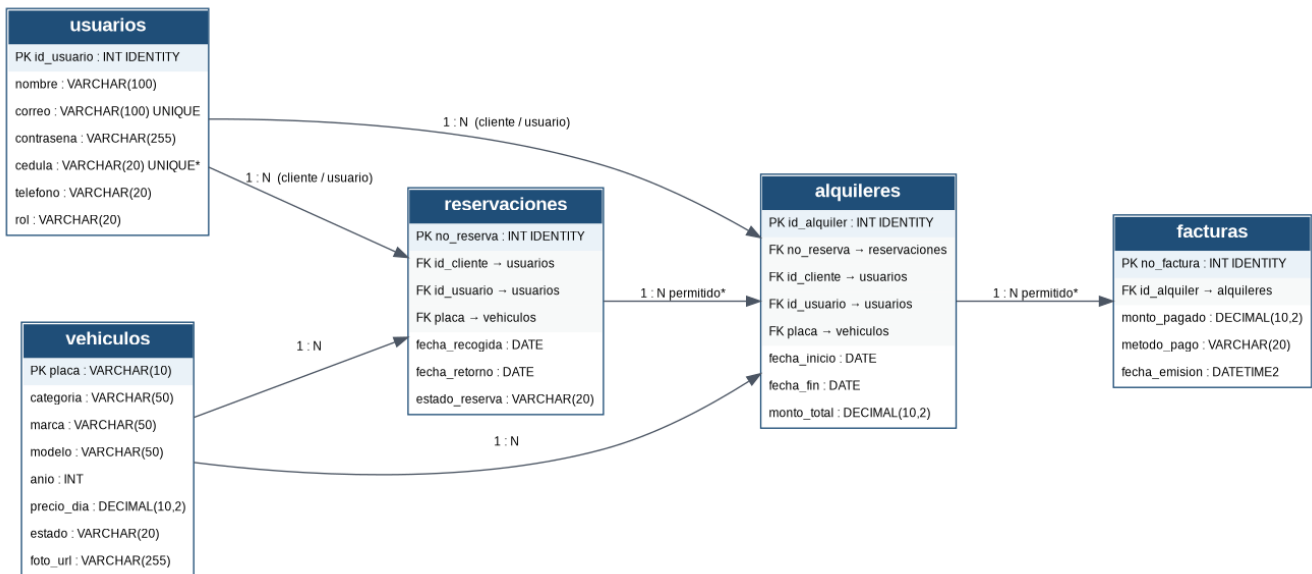


Figura 1. Diagrama Entidad–Relación de Sistema_ADV.

5.2 Diagrama funcional del proceso

El flujo funcional resume cómo una reservación puede originar un alquiler y cómo los triggers actúan después de insertar el alquiler para actualizar el estado del vehículo y calcular el monto total antes de registrar el pago.

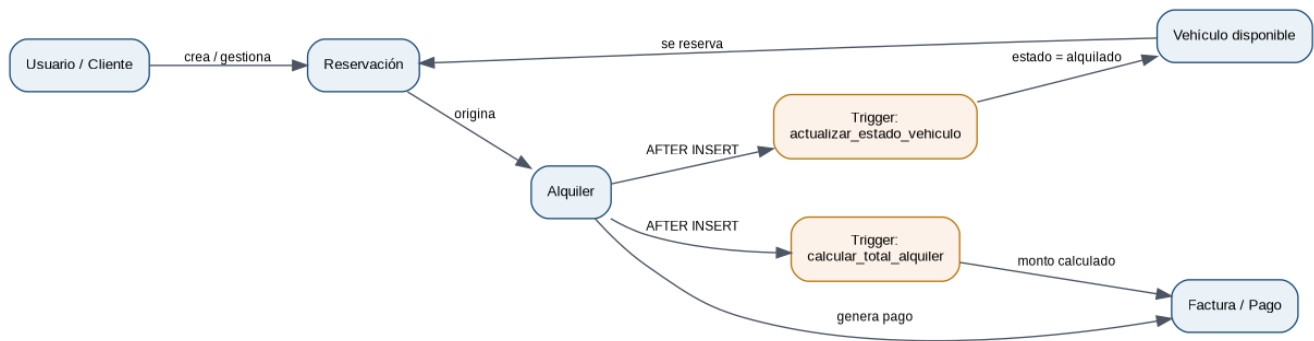


Figura 2. Flujo funcional principal del sistema.

6. Restricciones e integridad de datos

PRIMARY KEY: Identifica de forma única cada registro. Se utiliza en usuarios, vehículos, reservaciones, alquileres y facturas.

FOREIGN KEY: Mantiene la integridad entre entidades. Por ejemplo, una reservación no puede usar una placa que no exista en vehículos.

UNIQUE: Evita duplicados. correo es único y cedula se protege con un índice único filtrado cuando no es NULL.

NOT NULL: Obliga a proporcionar datos esenciales como nombre, correo, placa, fechas y precio_día.

DEFAULT: Asigna valores automáticos cuando no se especifican, como rol = Cliente, estado = disponible, estado_reserva = pendiente y fecha_emision = SYSDATETIME().

CHECK: Restringe dominios de valores: roles válidos, estados del vehículo, estados de reservación y métodos de pago.

6.1 Observaciones técnicas del esquema actual

- No existe una restricción que obligue a que fecha_fin o fecha_retorno sea igual o posterior a la fecha inicial.
- precio_día y monto_pagado no tienen una restricción CHECK que obligue a utilizar valores positivos.
- facturas.id_alquiler no es UNIQUE, por lo que se pueden registrar varios pagos o facturas asociados a un mismo alquiler.
- alquileres.no_reserva admite NULL y tampoco es UNIQUE; por tanto, el alquiler puede existir sin reserva previa y una reserva podría ser referenciada por más de un alquiler.
- La columna contraseña define la capacidad de almacenamiento, pero la seguridad del hash o cifrado corresponde a la capa de aplicación.

7. Vistas

Las vistas presentan consultas preparadas para obtener información frecuente sin repetir manualmente los JOIN y filtros en cada consulta.

7.1 vista_alquileres

Combina alquileres con usuarios y vehículos para mostrar el cliente, marca, modelo, fechas y monto total.

Ejemplo de consulta:

```
SELECT * FROM vista_alquileres;
```

7.2 vista_vehiculos_disponibles

Muestra únicamente los vehículos cuyo estado es disponible, junto con placa, marca, modelo y precio diario.

Ejemplo de consulta:

```
SELECT * FROM vista_vehiculos_disponibles;
```

7.3 vista_pagos

Relaciona facturas, alquileres y usuarios para presentar los pagos con el nombre del cliente, monto, método y fecha de emisión.

Ejemplo de consulta:

```
SELECT * FROM vista_pagos;
```

8. Triggers

Los triggers se ejecutan automáticamente después de insertar registros en alquileres. Ambos utilizan la tabla lógica inserted, por lo que pueden trabajar con uno o varios registros insertados en una misma operación.

8.1 actualizar_estado_vehiculo

Evento: AFTER INSERT sobre alquileres. Su función es localizar cada placa insertada y cambiar el estado correspondiente en vehiculos a alquilado. Así, el vehículo deja de aparecer como disponible inmediatamente después de crearse el alquiler.

```
CREATE TRIGGER actualizar_estado_vehiculo
ON alquileres
AFTER INSERT
AS
BEGIN
    UPDATE v
    SET v.estado = 'alquilado'
    FROM vehiculos AS v
    INNER JOIN inserted AS i ON v.placa = i.placa;
END;
```

8.2 calcular_total_alquiler

Evento: AFTER INSERT sobre alquileres. Calcula monto_total multiplicando la cantidad de días entre fecha_inicio y fecha_fin por precio_dia del vehículo. En la prueba principal, cuatro días a RD\$2,500 producen RD\$10,000.

```
CREATE TRIGGER calcular_total_alquiler
ON alquileres
AFTER INSERT
AS
BEGIN
    UPDATE a
    SET a.monto_total =
        DATEDIFF(DAY, a.fecha_inicio, a.fecha_fin) * v.precio_dia
    FROM alquileres AS a
    INNER JOIN inserted AS i ON a.id_alquiler = i.id_alquiler
    INNER JOIN vehiculos AS v ON a.placa = v.placa;
END;
```

9. Stored Procedures

Los procedimientos almacenados encapsulan operaciones frecuentes y permiten ejecutarlas mediante EXEC desde SSMS o desde una aplicación conectada a SQL Server.

9.1 registrar_cliente

Recibe nombre, correo, contraseña y teléfono. Inserta el registro en usuarios y asigna automáticamente el rol Cliente.

Ejemplo de ejecución:

```
EXEC registrar_cliente
  @nombre = 'Juan Perez',
  @correo = 'juan@gmail.com',
  @contrasena = '123456',
  @telefono = '8095551234';
```

9.2 vehiculos_disponibles

No requiere parámetros. Devuelve todos los vehículos cuyo estado es disponible.

Ejemplo de ejecución:

```
EXEC vehiculos_disponibles;
```

9.3 alquileres_por_cliente

Recibe @id_cliente y devuelve los alquileres asociados a ese usuario.

Ejemplo de ejecución:

```
EXEC alquileres_por_cliente @id_cliente = 1;
```

10. Script completo de creación

El siguiente script contiene la definición utilizada para crear Sistema_ADV en Microsoft SQL Server: base de datos, tablas, restricciones, vistas, triggers y procedimientos almacenados.

```

/* =====
SISTEMA DE GESTIÓN DE ALQUILER DE VEHÍCULOS
Base de datos: Sistema_ADV
Motor: Microsoft SQL Server
===== */

/* =====
1. CREACIÓN DE LA BASE DE DATOS
===== */

CREATE DATABASE Sistema_ADV;
GO

USE Sistema_ADV;
GO

/* =====
2. TABLA USUARIOS
===== */

CREATE TABLE usuarios (
    id_usuario INT IDENTITY(1,1) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(100) NOT NULL,
    contrasena VARCHAR(255) NOT NULL,
    cedula VARCHAR(20) NULL,
    telefono VARCHAR(20) NULL,
    rol VARCHAR(20) NOT NULL
        CONSTRAINT DF_usuarios_rol DEFAULT 'Cliente',

    CONSTRAINT UQ_usuarios_correo UNIQUE (correo),
    CONSTRAINT CK_usuarios_rol CHECK (rol IN ('Administrador', 'Cliente'))
);
GO

-- En SQL Server, una restricción UNIQUE normal solo permite un NULL.
-- Este índice conserva la lógica original: la cédula es opcional,
-- pero cuando se informa debe ser única.
CREATE UNIQUE INDEX UQ_usuarios_cedula
ON usuarios (cedula)
WHERE cedula IS NOT NULL;
GO

/* =====
3. TABLA VEHICULOS
===== */

CREATE TABLE vehiculos (
    placa VARCHAR(10) PRIMARY KEY,
    categoria VARCHAR(50) NULL,
    marca VARCHAR(50) NULL,
    modelo VARCHAR(50) NULL,
    anio INT NULL,
    precio_dia DECIMAL(10,2) NOT NULL,
    estado VARCHAR(20) NOT NULL
        CONSTRAINT DF_vehiculos_estado DEFAULT 'disponible',
    foto_url VARCHAR(255) NULL,

    CONSTRAINT CK_vehiculos_estado

```

```

        CHECK (estado IN ('disponible', 'alquilado', 'mantenimiento'))
    );
GO

/* =====
4. TABLA RESERVACIONES
===== */

CREATE TABLE reservaciones (
    no_reserva INT IDENTITY(1,1) PRIMARY KEY,
    id_cliente INT NOT NULL,
    id_usuario INT NOT NULL,
    placa VARCHAR(10) NOT NULL,
    fecha_recogida DATE NOT NULL,
    fecha_retorno DATE NOT NULL,
    estado_reserva VARCHAR(20) NOT NULL
        CONSTRAINT DF_reservaciones_estado DEFAULT 'pendiente',

    CONSTRAINT CK_reservaciones_estado
        CHECK (estado_reserva IN ('pendiente', 'confirmada', 'cancelada')),

    CONSTRAINT FK_reservaciones_cliente
        FOREIGN KEY (id_cliente) REFERENCES usuarios(id_usuario),

    CONSTRAINT FK_reservaciones_usuario
        FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),

    CONSTRAINT FK_reservaciones_vehiculo
        FOREIGN KEY (placa) REFERENCES vehiculos(placa)
);
GO

/* =====
5. TABLA ALQUILERES
===== */

CREATE TABLE alquileres (
    id_alquiler INT IDENTITY(1,1) PRIMARY KEY,
    no_reserva INT NULL,
    id_cliente INT NOT NULL,
    id_usuario INT NOT NULL,
    placa VARCHAR(10) NOT NULL,
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE NOT NULL,
    monto_total DECIMAL(10,2) NULL,

    CONSTRAINT FK_alquileres_reservacion
        FOREIGN KEY (no_reserva) REFERENCES reservaciones(no_reserva),

    CONSTRAINT FK_alquileres_cliente
        FOREIGN KEY (id_cliente) REFERENCES usuarios(id_usuario),

    CONSTRAINT FK_alquileres_usuario
        FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),

    CONSTRAINT FK_alquileres_vehiculo
        FOREIGN KEY (placa) REFERENCES vehiculos(placa)
);
GO

/* =====
6. TABLA FACTURAS
===== */

```

```

CREATE TABLE facturas (
    no_factura INT IDENTITY(1,1) PRIMARY KEY,
    id_alquiler INT NOT NULL,
    monto_pagado DECIMAL(10,2) NOT NULL,
    metodo_pago VARCHAR(20) NULL,
    fecha_emision DATETIME2 NOT NULL
        CONSTRAINT DF_facturas_fecha DEFAULT SYSDATETIME(),

    CONSTRAINT CK_facturas_metodo_pago
        CHECK (metodo_pago IN ('efectivo', 'tarjeta', 'transferencia')),

    CONSTRAINT FK_facturas_alquiler
        FOREIGN KEY (id_alquiler) REFERENCES alquileres(id_alquiler)
);
GO

```

```

/* =====
7. VISTAS
===== */

```

```

CREATE VIEW vista_alquileres
AS
SELECT
    a.id_alquiler,
    c.nombre AS cliente,
    v.marca,
    v.modelo,
    a.fecha_inicio,
    a.fecha_fin,
    a.monto_total
FROM alquileres AS a
INNER JOIN usuarios AS c ON a.id_cliente = c.id_usuario
INNER JOIN vehiculos AS v ON a.placa = v.placa;
GO

```

```

CREATE VIEW vista_vehiculos_disponibles
AS
SELECT
    placa,
    marca,
    modelo,
    precio_dia
FROM vehiculos
WHERE estado = 'disponible';
GO

```

```

CREATE VIEW vista_pagos
AS
SELECT
    f.no_factura,
    u.nombre AS cliente,
    f.monto_pagado,
    f.metodo_pago,
    f.fecha_emision
FROM facturas AS f
INNER JOIN alquileres AS a ON f.id_alquiler = a.id_alquiler
INNER JOIN usuarios AS u ON a.id_cliente = u.id_usuario;
GO

```

```

/* =====
8. TRIGGERS
===== */

```

```

CREATE TRIGGER actualizar_estado_vehiculo
ON alquileres

```

```

AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE v
    SET v.estado = 'alquilado'
    FROM vehiculos AS v
    INNER JOIN inserted AS i ON v.placa = i.placa;
END;
GO

CREATE TRIGGER calcular_total_alquiler
ON alquileres
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE a
    SET a.monto_total =
        DATEDIFF(DAY, a.fecha_inicio, a.fecha_fin) * v.precio_dia
    FROM alquileres AS a
    INNER JOIN inserted AS i ON a.id_alquiler = i.id_alquiler
    INNER JOIN vehiculos AS v ON a.placa = v.placa;
END;
GO

/* =====
9. STORED PROCEDURES
===== */

CREATE PROCEDURE registrar_cliente
    @nombre VARCHAR(100),
    @correo VARCHAR(100),
    @contrasena VARCHAR(255),
    @telefono VARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO usuarios (
        nombre, correo, contrasena, telefono, rol
    )
    VALUES (
        @nombre, @correo, @contrasena, @telefono, 'Cliente'
    );
END;
GO

CREATE PROCEDURE vehiculos_disponibles
AS
BEGIN
    SET NOCOUNT ON;

    SELECT *
    FROM vehiculos
    WHERE estado = 'disponible';
END;
GO

CREATE PROCEDURE alquileres_por_cliente
    @id_cliente INT
AS
BEGIN
    SET NOCOUNT ON;

```

```

SELECT *
FROM alquileres
WHERE id_cliente = @id_cliente;
END;
GO

```

11. Scripts de prueba

El archivo de pruebas está diseñado para ejecutarse después del script de creación sobre una base recién creada y vacía. Primero realiza operaciones válidas para comprobar el funcionamiento normal y, posteriormente, ejecuta casos inválidos dentro de bloques TRY/CATCH para verificar que las restricciones rechazan los datos incorrectos.

- 1. Registro de cliente.** Ejecuta registrar_cliente para crear a Juan Perez y comprueba usuarios.
- 2. Creación de administrador.** Inserta un usuario con rol Administrador.
- 3. Registro de vehículos.** Inserta un Toyota Corolla y un Honda CR-V como disponibles.
- 4. Consulta de disponibilidad.** Ejecuta vehiculos_disponibles y consulta vista_vehiculos_disponibles.
- 5. Creación de reservación.** Registra una reservación confirmada para el Toyota Corolla.
- 6. Creación de alquiler.** Inserta el alquiler sin monto_total para verificar el cálculo automático.
- 7. Comprobación de triggers.** Valida que el total sea RD\$10,000 y que el vehículo pase a estado alquilado.
- 8. Registro de factura.** Registra un pago de RD\$10,000 mediante tarjeta.
- 9. Consulta de vistas y procedimiento.** Consulta las tres vistas y alquileres_por_cliente.
- 10. Pruebas negativas.** Intenta insertar datos inválidos y comprueba que SQL Server los rechace.

12. Pruebas de integridad

Las pruebas negativas no representan fallos del sistema. En estos casos, recibir un error de SQL Server es el resultado esperado, porque demuestra que las restricciones y claves foráneas están activas.

Correo duplicado: La restricción UNIQUE de usuarios.correo debe rechazar un segundo registro con juan@gmail.com.

Rol inválido: CK_usuarios_rol debe rechazar el valor Empleado.

Estado de vehículo inválido: CK_vehiculos_estado debe rechazar vendido.

Vehículo inexistente: FK_reservaciones_vehiculo debe impedir reservar la placa Z999999.

Cliente inexistente: FK_reservaciones_cliente debe impedir utilizar id_cliente = 9999.

Estado de reservación inválido: CK_reservaciones_estado debe rechazar finalizada.

Reservación inexistente: FK_alquileres_reservacion debe impedir usar no_reserva = 9999.

Método de pago inválido: CK_facturas_metodo_pago debe rechazar cheque.

13. Resultados esperados de las pruebas

Prueba	Resultado esperado	Criterio de validación
Registrar cliente	Usuario creado con rol Cliente	Correcto si aparece en usuarios
Registrar administrador	Usuario con rol Administrador	Correcto si se inserta
Registrar vehículos	Dos vehículos disponibles	Correcto si ambos aparecen
Consultar disponibilidad	A123456 y B654321 antes del alquiler	Correcto
Crear reservación	Reserva confirmada no. 1	Correcto si se inserta
Crear alquiler	monto_total = RD\$10,000	Correcto si trigger calcula $4 \times 2,500$
Estado del vehículo	A123456 = alquilado	Correcto si trigger actualiza

Prueba	Resultado esperado	Criterio de validación
Registrar factura	Pago RD\$10,000 por tarjeta	Correcto si se inserta
Correo duplicado	Rechazado	Prueba aprobada si genera error
Rol inválido	Rechazado	Prueba aprobada si genera error
Estado vehicular inválido	Rechazado	Prueba aprobada si genera error
Placa inexistente	Rechazada	Prueba aprobada si genera error
Cliente inexistente	Rechazado	Prueba aprobada si genera error
Estado reserva inválido	Rechazado	Prueba aprobada si genera error
Reserva inexistente	Rechazada	Prueba aprobada si genera error
Método de pago inválido	Rechazado	Prueba aprobada si genera error

14. Script completo de pruebas

A continuación se incluye el script íntegro utilizado para validar el funcionamiento de Sistema_ADV y sus reglas de integridad.

```

/* =====
SISTEMA_ADV - SCRIPT DE PRUEBAS GENERALES
Motor: Microsoft SQL Server

REQUISITO:
Ejecutar primero Sistema_ADV_SQLServer.sql.

NOTA:
Este archivo está diseñado para una base recién creada y vacía.
Incluye pruebas funcionales y pruebas de integridad.
Algunas pruebas negativas DEBEN producir errores: eso significa
que las restricciones están funcionando correctamente.
===== */

USE Sistema_ADV;
GO

/* =====
1. REGISTRAR CLIENTE MEDIANTE STORED PROCEDURE
===== */

EXEC registrar_cliente
    @nombre = 'Juan Perez',
    @correo = 'juan@gmail.com',
    @contrasena = '123456',
    @telefono = '8095551234';
GO

SELECT * FROM usuarios;
GO

/* =====
2. CREAR ADMINISTRADOR
===== */

INSERT INTO usuarios (
    nombre, correo, contrasena, cedula, telefono, rol
)
VALUES (
    'Administrador Sistema',
    'admin@sistema.com',

```

```

        'Admin123',
        '00100000001',
        '8095550000',
        'Administrador'
    );
GO

```

```

/* =====
3. REGISTRAR VEHÍCULOS
===== */

```

```

INSERT INTO vehiculos (
    placa, categoria, marca, modelo, anio,
    precio_dia, estado, foto_url
)
VALUES
('A123456', 'Sedan', 'Toyota', 'Corolla', 2024, 2500.00, 'disponible', NULL),
('B654321', 'SUV', 'Honda', 'CR-V', 2023, 3500.00, 'disponible', NULL);
GO

```

```

SELECT * FROM vehiculos;
GO

```

```

/* =====
4. PROBAR VEHÍCULOS DISPONIBLES
===== */

```

```

EXEC vehiculos_disponibles;
GO

```

```

SELECT * FROM vista_vehiculos_disponibles;
GO

```

```

/* =====
5. CREAR RESERVACIÓN
Cliente: ID 1
Administrador: ID 2
Vehículo: A123456
===== */

```

```

INSERT INTO reservaciones (
    id_cliente, id_usuario, placa,
    fecha_recogida, fecha_retorno, estado_reserva
)
VALUES (
    1, 2, 'A123456',
    '2026-10-01', '2026-10-05', 'confirmada'
);
GO

```

```

SELECT * FROM reservaciones;
GO

```

```

/* =====
6. CREAR ALQUILER
monto_total se omite intencionalmente.
El trigger debe calcular:
4 días x 2,500.00 = 10,000.00
===== */

```

```

INSERT INTO alquileres (
    no_reserva, id_cliente, id_usuario,

```

```

    placa, fecha_inicio, fecha_fin
)
VALUES (
    1, 1, 2,
    'A123456', '2026-10-01', '2026-10-05'
);
GO

SELECT * FROM alquileres;
GO

/* =====
7. COMPROBAR TRIGGER DE ESTADO DEL VEHÍCULO

Resultado esperado para A123456: alquilado
===== */

SELECT placa, marca, modelo, estado
FROM vehiculos
WHERE placa = 'A123456';
GO

EXEC vehiculos_disponibles;
GO

/* =====
8. REGISTRAR FACTURA
===== */

INSERT INTO facturas (
    id_alquiler, monto_pagado, metodo_pago
)
VALUES (
    1, 10000.00, 'tarjeta'
);
GO

SELECT * FROM facturas;
GO

/* =====
9. PROBAR LAS TRES VISTAS
===== */

SELECT * FROM vista_alquileres;
GO

SELECT * FROM vista_vehiculos_disponibles;
GO

SELECT * FROM vista_pagos;
GO

/* =====
10. PROBAR ALQUILERES POR CLIENTE
===== */

EXEC alquileres_por_cliente
    @id_cliente = 1;
GO

/* =====
```

11. PRUEBAS DE INTEGRIDAD

ATENCIÓN:

Cada bloque siguiente debe ser rechazado por SQL Server.
 Los errores esperados confirman que las restricciones funcionan.
 ===== */

/* PRUEBA NEGATIVA 1 - CORREO DUPLICADO */

```
BEGIN TRY
    INSERT INTO usuarios (
        nombre, correo, contrasena, telefono, rol
    )
    VALUES (
        'Usuario Duplicado',
        'juan@gmail.com',
        '123456',
        '8095559999',
        'Cliente'
    );

    PRINT 'ERROR DE PRUEBA: se permitió un correo duplicado.';
END TRY
BEGIN CATCH
    PRINT 'PRUEBA APROBADA - correo duplicado rechazado.';
    PRINT ERROR_MESSAGE();
END CATCH;
GO
```

/* PRUEBA NEGATIVA 2 - ROL INVÁLIDO */

```
BEGIN TRY
    INSERT INTO usuarios (
        nombre, correo, contrasena, telefono, rol
    )
    VALUES (
        'Usuario Prueba',
        'prueba@sistema.com',
        '123456',
        '8095558888',
        'Empleado'
    );

    PRINT 'ERROR DE PRUEBA: se permitió un rol inválido.';
END TRY
BEGIN CATCH
    PRINT 'PRUEBA APROBADA - rol inválido rechazado.';
    PRINT ERROR_MESSAGE();
END CATCH;
GO
```

/* PRUEBA NEGATIVA 3 - ESTADO DE VEHÍCULO INVÁLIDO */

```
BEGIN TRY
    INSERT INTO vehiculos (
        placa, categoria, marca, modelo,
        anio, precio_dia, estado
    )
    VALUES (
        'C111111', 'SUV', 'Hyundai', 'Tucson',
        2025, 4000.00, 'vendido'
    );

    PRINT 'ERROR DE PRUEBA: se permitió un estado de vehículo inválido.';
END TRY
```

```

BEGIN CATCH
    PRINT 'PRUEBA APROBADA - estado de vehículo inválido rechazado.';
    PRINT ERROR_MESSAGE();
END CATCH;
GO

/* PRUEBA NEGATIVA 4 - VEHÍCULO INEXISTENTE EN RESERVACIÓN */

BEGIN TRY
    INSERT INTO reservaciones (
        id_cliente, id_usuario, placa,
        fecha_recogida, fecha_retorno, estado_reserva
    )
    VALUES (
        1, 2, 'Z999999',
        '2026-11-01', '2026-11-05', 'confirmada'
    );

    PRINT 'ERROR DE PRUEBA: se permitió una placa inexistente.';
END TRY
BEGIN CATCH
    PRINT 'PRUEBA APROBADA - placa inexistente rechazada.';
    PRINT ERROR_MESSAGE();
END CATCH;
GO

/* PRUEBA NEGATIVA 5 - CLIENTE INEXISTENTE */

BEGIN TRY
    INSERT INTO reservaciones (
        id_cliente, id_usuario, placa,
        fecha_recogida, fecha_retorno, estado_reserva
    )
    VALUES (
        9999, 2, 'B654321',
        '2026-11-01', '2026-11-05', 'confirmada'
    );

    PRINT 'ERROR DE PRUEBA: se permitió un cliente inexistente.';
END TRY
BEGIN CATCH
    PRINT 'PRUEBA APROBADA - cliente inexistente rechazado.';
    PRINT ERROR_MESSAGE();
END CATCH;
GO

/* PRUEBA NEGATIVA 6 - ESTADO DE RESERVACIÓN INVÁLIDO */

BEGIN TRY
    INSERT INTO reservaciones (
        id_cliente, id_usuario, placa,
        fecha_recogida, fecha_retorno, estado_reserva
    )
    VALUES (
        1, 2, 'B654321',
        '2026-11-01', '2026-11-05', 'finalizada'
    );

    PRINT 'ERROR DE PRUEBA: se permitió un estado de reservación inválido.';
END TRY
BEGIN CATCH
    PRINT 'PRUEBA APROBADA - estado de reservación inválido rechazado.';
    PRINT ERROR_MESSAGE();
END CATCH;

```

GO

```

/* PRUEBA NEGATIVA 7 - RESERVACIÓN INEXISTENTE EN ALQUILER */

BEGIN TRY
    INSERT INTO alquileres (
        no_reserva, id_cliente, id_usuario,
        placa, fecha_inicio, fecha_fin
    )
    VALUES (
        9999, 1, 2,
        'B654321', '2026-11-01', '2026-11-05'
    );

    PRINT 'ERROR DE PRUEBA: se permitió una reservación inexistente.';
END TRY
BEGIN CATCH
    PRINT 'PRUEBA APROBADA - reservación inexistente rechazada.';
    PRINT ERROR_MESSAGE();
END CATCH;
GO

```

```

/* PRUEBA NEGATIVA 8 - MÉTODO DE PAGO INVÁLIDO */

BEGIN TRY
    INSERT INTO facturas (
        id_alquiler, monto_pagado, metodo_pago
    )
    VALUES (
        1, 10000.00, 'cheque'
    );

    PRINT 'ERROR DE PRUEBA: se permitió un método de pago inválido.';
END TRY
BEGIN CATCH
    PRINT 'PRUEBA APROBADA - método de pago inválido rechazado.';
    PRINT ERROR_MESSAGE();
END CATCH;
GO

```

```

/* =====
12. ESTADO FINAL DE LOS DATOS
===== */

SELECT * FROM usuarios;
SELECT * FROM vehiculos;
SELECT * FROM reservaciones;
SELECT * FROM alquileres;
SELECT * FROM facturas;
GO

PRINT 'FIN DE LAS PRUEBAS DE Sistema_ADV';
GO

```

15. Conclusión técnica

Sistema_ADV contiene una estructura relacional compuesta por cinco tablas y complementada con vistas, triggers y procedimientos almacenados. Las claves foráneas mantienen la relación entre usuarios, vehículos, reservaciones, alquileres y facturas; las restricciones CHECK y UNIQUE controlan valores válidos y duplicados; y los triggers automatizan el cambio de estado del vehículo y el cálculo del total del alquiler.

El script de pruebas permite verificar tanto el comportamiento normal como la protección ante datos inválidos. En una ejecución sobre una base recién creada, los casos funcionales deben completar sus operaciones y

los casos negativos deben ser rechazados por SQL Server. De esta forma, el documento reúne la estructura, funcionamiento, relaciones, diagramas, scripts y criterios de validación de la base de datos.